
LemmaPortfolio: Choosing One Small Lemma Set for Many Lean Goals

Aishwarya Praveen Das
Dirac Labs, Madison, USA

Durgesh Kumar
Dirac Labs, Madison, USA

Abstract

Premise selection normally asks which lemmas may help one proof state. We study the shared-budget problem: which small lemma set best serves several related goals at once? A LEMMAPORTFOLIO episode displays the types of 16 name-masked candidates and eight name-masked targets, and asks for three candidates. A target is covered when at least one chosen candidate occurs directly in its historical elaborated value. We construct 30 development and 60 test episodes from MATHLIB v4.33.0, enumerate all 560 portfolios, retain every maximizer, and independently replay the dependency labels in pinned Lean. The public test set and label digest were committed before the response document was supplied. Among five supplied consumer-response rows, exact-optimum accuracy ranged from 1/60 to 25/60; the best row covered 299/480 targets but attained an optimum on only 41.7% of episodes. On 20 matched items, a support-prediction decomposition attained 1/20 exact optima, compared with 4/20 under direct prompting, locating a concrete failure mode rather than an improvement. The task measures compression of historical dependencies, not proof success under added premises.

1 Introduction: one budget, several goals

Suppose that a prover can place only three lemmas in a shared context for eight related goals. Choosing a plausible lemma for each goal independently can waste that budget: a less obvious lemma may be useful across several goals. The same choice arises when building a retrieval cache for a file or asking a mathematician to inspect a short list of reusable lemmas. We call this shared selection problem the *lemma-portfolio* problem.

Most premise retrievers are queried for one proof state. LeanDojo and ReProver make this setting explicit for Lean [Yang et al., 2023], Magnushammer studies the analogous problem for Isabelle [Mikuła et al., 2024], and recent Lean work also uses the dependency graph [Petrovčič et al., 2025]; LeanHammer adapts neural selection to local contexts and couples it to proof reconstruction [Zhu et al., 2026]. UseFor asks the stronger utility question of how a theorem changes proof likelihood [Kasriel et al., 2025]. LeanSearch v2 retrieves a premise group for one whole theorem [Gao et al., 2026], while miniCTX studies theorem proving with long project context [Hu et al., 2025]. TheoremBench expands one theorem into related proving tasks and reports partial progress [Pham et al., 2026], but does not impose one shared premise budget across them. We instead isolate the finite decision that comes before several proofs: one fixed-size portfolio from a closed pool must serve all of them. To our knowledge, the formal-premise benchmarks reviewed above do not score this multi-target, shared-budget objective with complete exact optima.

The system must infer a hidden pattern of historical reuse from formal statements alone. Because the pool is small, we can enumerate every portfolio and credit every maximizer rather than privilege one answer. Our contribution is threefold: a precise shared-budget objective, a module-disjoint and exactly audited MATHLIB benchmark, and an evaluation that separates exact portfolio choice from

partial coverage and support prediction. The benchmark is not a new prover or a claim about logical necessity.

2 Task, scoring, and construction

An episode contains a candidate pool C with $|C| = 16$, a target set T with $|T| = 8$, and a budget $B = 3$. Candidate and target declaration names are replaced by identifiers C01–C16 and T01–T08; candidate types and target statements remain visible. We use “lemma” informally for a candidate declaration, which may also be a definition or constructor. For a target t , let

$$D(t) = \{c \in C : c \text{ occurs as a direct constant in the elaborated value of } t\}.$$

Occurrence is syntactic in the elaborated declaration body; we take no transitive closure. For a theorem, this body is its elaborated proof term. For a portfolio $S \subseteq C$ of size B , its historical target coverage is

$$F(S) = |\{t \in T : D(t) \cap S \neq \emptyset\}|. \quad (1)$$

Let $F^* = \max_{|S|=B} F(S)$ and $\mathcal{A} = \{S : |S| = B, F(S) = F^*\}$. We enumerate all $\binom{16}{3} = 560$ portfolios and reconstruct $\mathcal{A}$. The primary test score is the number n of the 60 episodes for which the submitted portfolio is in $\mathcal{A}$, reported as $n/60$ and a percentage. Thus every maximizing portfolio is correct. The secondary item scores are raw target coverage $F(S)$ (also $F(S)/8$), normalized coverage $F(S)/F^*$, and direct-edge recall

$$R_E(S) = \frac{\sum_{t \in T} |D(t) \cap S|}{\sum_{t \in T} |D(t)|}.$$

Aggregate edge recall is micro-averaged by summing this numerator and denominator over episodes. A missing, malformed, wrong-budget, duplicate-ID, or out-of-pool response gets zero on all four scores and remains in the 60-item denominator; validity is reported separately.

This is a small maximum-coverage problem [Nemhauser et al., 1978], but the incidence graph is hidden behind the displayed statements. If C01 serves three targets, C02 two others, and C03 a sixth, their portfolio covers six; spending two slots on alternatives for the first three targets can waste the budget. The 16/8/3 design keeps prompts readable and all choices enumerable while still rewarding reuse.

Source and construction. We mine target values and candidate types from MATHLIB v4.33.0 at commit db584cd6d46c92f209a44c0f1c829460d327499d [The mathlib Community, 2020, 2026]. Each episode comes from one module, with names hidden. Development, test, and all earlier prototype modules and statements are disjoint. We reject simple “largest star” instances: each retained episode has at least seven active candidates, at least four active candidates outside every optimum, multiple multi-edge targets, and multiple near-optimal portfolios; every member of an optimum is reused and essential to that portfolio. The complete thresholds are in the supplement.

The miner also rejects any episode solved by any of 20 fixed construction methods. Eighteen use only public strings or positions; two use the true incidence graph and rank candidates by frequency or greedy maximum coverage. These methods define the challenge distribution and are not held-out baselines. Among the 60 test episodes, 45 have $F^* = 6$ and 15 have $F^* = 7$; together they have 78 accepted optimal portfolios. The final split contains 30 development and 60 test episodes across 15 top-level domains.

Commitment and label audit. The underlying theorem statements are public and may occur in model training, so we do not claim contamination-free evaluation. What is new is their composition into name-masked episodes and the hidden incidence rows. On 1 September 2026 we recorded the public-file hash and a digest of the label file; the response PDF used below was supplied on 2 September. The prompts do not contain the labels. A separate pinned-Lean 4.33.0 replay reconstructed every displayed edge, and scoring freshly reconstructs every optimum. The response PDF lacks provider logs and per-chat timestamps, so the deployment rows are external captures with a prior dataset commitment, not a cryptographic blind evaluation. The supplement states this evidence boundary precisely.

Table 1: V4 test results. The primary score credits an episode only when the selected triple attains the exact maximum coverage; mean target coverage gives partial credit. Both use the fixed 60-episode denominator.

System or deployment label	Exact optimum	Mean coverage	
	$n/60$ (%)	$F(S)/8$ (%)	Valid
Uniform random (analytic expectation)	0.14/60 (0.2)	2.35/8 (29.3)	–
Character TF-IDF	0/60 (0.0)	3.77/8 (47.1)	60/60
GPT SOL 5.6 Pro	25/60 (41.7)	4.98/8 (62.3)	55/60
DeepSeek Instant + DeepThink	1/60 (1.7)	4.20/8 (52.5)	60/60
DeepSeek Expert + DeepThink	7/60 (11.7)	4.60/8 (57.5)	60/60
Qwen 3.8 Max-Thinking	12/60 (20.0)	4.32/8 (54.0)	55/60
Qwen 3.7 Plus-Thinking	6/60 (10.0)	4.05/8 (50.6)	60/60

3 Experimental setup

We ask three questions. How often does a displayed deployment select an exact optimum? How much of the target set does it cover when it misses? Finally, does making the hidden target–candidate relation explicit improve the joint decision? The author-supplied document reports five visible consumer labels: GPT SOL 5.6 Pro, two DeepSeek DeepThink modes, and Qwen 3.8 Max and 3.7 Plus in Thinking mode. Each direct row is intended to cover all 60 episodes in twelve five-item prompts. We align responses by their explicit episode IDs, not by page position, preserve malformed or missing blocks as zeroes, and make no repair. The analysis retains at most one supplied capture per intended block; anomalies remain zero-scored, and no repeated sampling is available.

The sole model-facing development check used the first 15 development episodes: a controlled Codex CLI Sol capture obtained 6/15 exact optima. It motivated no change to the committed test set and is not pooled with the consumer rows.

Consumer interfaces do not expose the serving checkpoint, system prompt, decoding policy, or routing decisions. We therefore report the visible labels verbatim. The convenience sample is neither representative nor a ranking of providers or model families.

Two non-model references were fixed in advance. Under uniform selection, every portfolio has probability $1/560$, so the probability of an optimum is $|\mathcal{A}|/560$. If $d_t = |D(t)|$, the expected episode coverage is

$$\mathbb{E}[F(S)] = \sum_{t \in T} \left(1 - \frac{\binom{16-d_t}{3}}{\binom{16}{3}} \right),$$

and expected edge recall is $3/16$; no Monte Carlo estimate is needed. A character 3–5-gram TF-IDF reference instead enumerates all portfolios and maximizes summed target-to-candidate facility-location similarity. It was run only after the split commitment and never used to admit an item. The supplement gives the full calculations.

4 Results

The exact and partial-credit columns tell different parts of the story. GPT SOL had the largest exact score, 25/60, and covered 299 of the 480 displayed targets; its mean oracle-normalized coverage was 79.6%. Qwen 3.8 Max had the next largest exact score, 12/60. The remaining exact scores were 7/60 for DeepSeek Expert, 6/60 for Qwen 3.7 Plus, and 1/60 for DeepSeek Instant. Thus a system could often identify useful candidates without assembling a maximizing three-candidate portfolio. Character TF-IDF likewise covered 47.1% of targets but found no exact optimum, which indicates that the task is not solved by this surface-similarity rule.

Two five-item response blocks were invalid under strict whole-block parsing: the GPT document repeats the block for episodes 0050–0054 and supplies no direct block for 0055–0059, while the Qwen 3.8 block for 0055–0059 lacks its opening outer brace. We neither repair nor remove these outcomes. Episode keys, rather than the visual block order, align every other response to its prompt. Wilson and whole-block bootstrap intervals, per-block counts, and the full transcription audit appear in the supplement. These convenience-sample rows identify dated consumer labels, not reproducible checkpoints or model families.

5 Diagnosing errors with structured support prediction

A direct portfolio does not show whether Sol inferred the target–candidate relations incorrectly or combined plausible relations poorly. On 20 test episodes, four additional responses return up to four candidate IDs for each target, ordered by plausibility. A deterministic program keeps the first two per target ($q = 2$), then exactly solves the induced size-three coverage problem; ranks three and four are used only to diagnose the inferred incidence graph.

On the same 20 public episode IDs, direct GPT SOL prompting attained 4/20 exact optima, whereas ordered support prediction followed by the fixed $q = 2$ optimizer attained 1/20, a descriptive change of -15 percentage points. The retained support edges nevertheless had 76.8% micro precision and 68.7% micro recall. This particular decomposition therefore produced a reasonably informative incidence sketch but a worse exact portfolio; it does not support an improvement claim.

This is an error-localization diagnostic, not a second model. It compares direct prompting with the complete structured-prompt-plus-solver pipeline. We report item and whole-block differences without a significance test. Since the source document does not certify paired-call chronology, the contrast is descriptive rather than causal.

6 Limitations and ways forward

LEMMAPORTFOLIO compresses one historical direct-dependency graph. Direct occurrence is reproducible, but it is neither logical necessity nor evidence that supplying a lemma will cause a new proof to succeed. The pool is closed, all costs are uniform, and the 60 modules form a deliberately conditioned stress set. Public statements may have appeared in training. Consumer routing is opaque, the response document supplies no per-call operational evidence, and we have one pass and no human-expert baseline. Intervals therefore describe this fixed set; they do not estimate performance over MATHLIB or generation variance.

The natural next step is to train and evaluate set-level retrievers rather than assemble per-goal rankings after the fact. Predicted incidence can still be useful, but the $q = 2$ result shows that calibration, truncation, and tie-breaking must be studied jointly. Larger candidate pools, nonuniform context costs, causal proof-success labels, repeated runs, and a mathematician baseline would move the benchmark closer to prover practice.

7 Conclusion

Choosing one shared budget across several goals is a finite decision distinct from per-goal retrieval, and it admits exact scoring with every tie preserved. The observed gap between partial coverage and exact optimality shows that recognizing individually plausible premises does not by itself solve the shared-budget decision. The supplement and anonymous artifact provide the data, released labels, response transcription, scorer, Lean replay receipts, and reproduction commands.

References

- Guoxiong Gao, Zeming Sun, Jiedong Jiang, Yutong Wang, Jingda Xu, Peihao Wu, Bryan Dai, and Bin Dong. LeanSearch v2: Global premise retrieval for Lean 4 theorem proving, 2026. URL <https://arxiv.org/abs/2605.13137>.
- Jiewen Hu, Thomas Zhu, and Sean Welleck. miniCTX: Neural theorem proving with (long-)contexts. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=KIgaAqEFHW>.
- Timothe Kasriel, Thomas Lu, Devon Ding, Jingxuan He, and Dawn Song. Usefulness-driven learning of formal mathematics. In *NeurIPS Workshop on Mathematical Reasoning and AI*, 2025. URL <https://openreview.net/pdf/023eb56eadd5685c15a6489e0fddd540279b8c45.pdf>.
- Maciej Mikula, Szymon Tworkowski, Szymon Antoniak, Bartosz Piotrowski, Qiaochu Jiang, Jin Zhou, Christian Szegedy, Łukasz Kuciński, Piotr Miłoś, and Yuhuai Wu. Magnushammer: A transformer-based approach to premise selection. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/ab5f5f22e3e09f4424592ffb06840ab0-Abstract-Conference.html.
- George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. An analysis of approximations for maximizing submodular set functions—i. *Mathematical Programming*, 14:265–294, 1978. doi: 10.1007/BF01588971.
- Job Petrovič, David Eliecer Narvaez Denis, and Ljupčo Todorovski. Combining textual and structural information for premise selection in Lean. In *NeurIPS Workshop on Mathematical Reasoning and AI*, 2025. URL <https://arxiv.org/abs/2510.23637>.
- QuocViet Pham, Elvir Karimov, Andrey Galichin, and Ivan Oseledets. TheoremBench: Evaluating LLMs on theorem proving in formal mathematics, 2026. URL <https://arxiv.org/abs/2606.09450>.
- The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*. ACM, 2020. doi: 10.1145/3372885.3373824.
- The mathlib Community. mathlib4 source snapshot for version 4.33.0. Git repository commit, 2026. URL <https://github.com/leanprover-community/mathlib4>. Commit db584cd6d46c92f209a44c0f1c829460d327499d.
- Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023. doi: 10.52202/075280-0944. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/4441469427094f8873d0fecb0c4e1cee-Abstract-Datasets_and_Benchmarks.html.
- Thomas Zhu, Joshua Clune, Jeremy Avigad, Albert Qiaochu Jiang, and Sean Welleck. Premise selection for a Lean hammer. In *International Conference on Learning Representations*, 2026. doi: 10.48550/arXiv.2506.07477. URL <https://openreview.net/pdf/0d32353ccfa85cbf700389ec2d98a2c40fa4b640.pdf>.